

# LayerZero Onesig

Security Assessment

July 23rd, 2026 — Prepared by OtterSec


Andreas Mantzoutas andreas@osec.io

Renato Eugenio Maria Marziano renato@osec.io

# Table of Contents


**Executive Summary** **2**

Overview 2

Key Findings 2

**Scope** **3**

**Findings** **4**

**Vulnerabilities** **5**

OS-LZO-ADV-00 | Inconsistency in Executor Disable Flag Implementation 6

**General Findings** **7**

OS-LZO-SUG-00 | Divergence in Event Emission 8

OS-LZO-SUG-01 | Unrestricted Sub-Invocation Authorization 9

OS-LZO-SUG-02 | Root Invocation Binding Should Always Be Included 10

## Appendices

**Vulnerability Rating Scale** **11**

**Procedure** **12**


© 2026 Otter Audits LLC. All Rights Reserved.

# 01 — Executive Summary

## Overview

LayerZero engaged OtterSec to assess the `onesig` program. This assessment was conducted between February 25th and March 16th, 2026. Follow-up audits were conducted on May 7th and July 22nd, 2026. For more information on our auditing methodology, refer to Appendix B.

## Key Findings

We produced 4 findings throughout this audit engagement.

In particular, we identified an inconsistency in the sender verification logic, which continues to enforce sender-specific checks for Executor and Signer variants even when no executor is required, meaning execution is not truly permissionless in that mode (OS-LZO-ADV-00).

Additionally, we highlighted instances where the Stellar OneSig implementation diverges from the EVM version by allowing zero-valued addresses and by not emitting an event on setting the seed (OS-LZO-SUG-00). We also advised reevaluating the current implementation to determine whether sub invocations should allow arbitrary authorization entries without validation (OS-LZO-SUG-01). Lastly, we suggested including the root invocation in the signed payload, as otherwise the authorization may be consumed through a different top-level callee that reproduces the same downstream calls (OS-LZO-SUG-02).

© 2026 Otter Audits LLC. All Rights Reserved.

# 02 — Scope

The source code was delivered to us in a Git repository at https://github.com/LayerZero-Labs/monorepo-internal. This audit was initially performed against commit ebd661e. Follow-up audits were performed against ebd661e on LayerZero-Labs/audit-external, as well as against commits dcade8e and 0781f4e on LayerZero-Labs/monorepo-external.

**A brief description of the program is as follows:**

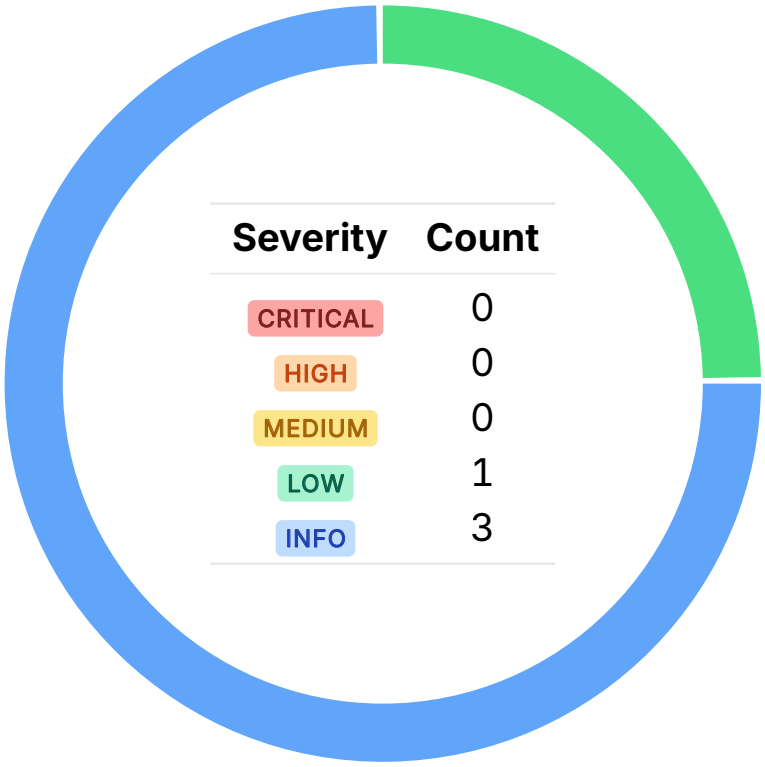
| Name | Description |
| --- | --- |
| onesig | It implements a Merkle-based multisig execution system for Soroban, where signers approve a Merkle root via EIP-712 signatures and individual transactions are later proven against that root for execution. |

© 2026 Otter Audits LLC. All Rights Reserved.

# 03 — Findings

Overall, we reported 4 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



| Severity | Count |
| --- | --- |
| CRITICAL | 0 |
| HIGH | 0 |
| MEDIUM | 0 |
| LOW | 1 |
| INFO | 3 |

© 2026 Otter Audits LLC. All Rights Reserved.

# 04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

| ID | Severity | Status | Description |
|---|---|---|---|
| OS-LZO-ADV-00 | LOW | RESOLVED | `verify_sender` still enforces sender-specific checks for `Executor` and `Signer` variants even when `executor_required` is disabled, so execution is not truly permissionless in that mode. |

© 2026 Otter Audits LLC. All Rights Reserved.

## Inconsistency in Executor Disable Flag Implementation LOW OS-LZO-ADV-00

### Description

`auth::verify_sender` does not fully honor `executor_required == false`, since it still validates `Executor` and `Signer` variants instead of immediately allowing execution. As a result, transactions may fail unnecessarily based on the provided `Sender` variant, even though execution is intended to be permissionless in that mode.

>_ *packages/onesig/onesig-stellar/contracts/onesig/src/auth.rs* RUST

```rust
/// Verifies the sender has permission to execute the transaction.
fn verify_sender(env: &Env, sender: &Sender, signature_payload: &BytesN<32>) {
    match sender {
        Sender::Executor(pubkey, signature) => {
            Self::verify_executor_permissions(env, pubkey, signature, signature_payload);
        }
        Sender::Signer(signature) => {
            Self::verify_signer_permissions(env, signature_payload, signature);
        }
        Sender::Permissionless => {
            assert_with_error!(
                env,
                !Self::executor_required(env),
                OneSigError::OnlyExecutorOrSigner
            );
        }
    }
}
```

### Remediation

Ensure that `verify_sender` immediately succeeds when `executor_required` is false.

### Patch

The team has addressed this in PR #3219.

© 2026 Otter Audits LLC. All Rights Reserved.

# 05 — General Findings

Here, we present a discussion of general findings identified during our audit. While these findings do not pose an immediate security impact, they represent anti-patterns and may result in security issues in the future.

| ID | Description |
| --- | --- |
| OS-LZO-SUG-00 | No event is emitted when calling `OneSigStorage::set_seed` in the constructor, unlike in the EVM implementation. |
| OS-LZO-SUG-01 | `sub_invocations` allows arbitrary authorization entries without validation, potentially granting downstream contracts unintended access to OneSig's authority. |
| OS-LZO-SUG-02 | If the root invocation is not included in the signed payload, authorization may be consumed through a different top-level callee that reproduces the same downstream calls. |

© 2026 Otter Audits LLC. All Rights Reserved.

## Divergence in Event Emission

OS-LZO-SUG-00

### Description

The Onesig implementation does not emit events when calling `OneSigStorage::set_seed` in the constructor, unlike the EVM implementation shown here:

>_ *packages/onesig/onesig-evm/contracts/OneSig.sol* SOLIDITY

```
function _setSeed(bytes32 _seed) internal virtual {
    seed = _seed;
    emit SeedSet(_seed);
}
```

### Remediation

Emit an event on setting the seed.

### Patch

The LayerZero team has verified that their existing off-chain systems do not rely on this event.

© 2026 Otter Audits LLC. All Rights Reserved.

## Unrestricted Sub-Invocation Authorization OS-LZO-SUG-01

### Description

`Call.sub_invocations` can include any variant of `InvokerContractAuthEntry` types, and the implementation does not validate or restrict which variants are allowed. Although tests assume only `Contract`-type authorization, other forms may be embedded and pre-authorized without explicit checks. This allows downstream contracts to potentially act with OneSig's authority to deploy contracts. Additionally, this authority can only be granted only to specified subcalls, not the direct callee.

### Remediation

Reevaluate the current implementation to determine whether this behavior is intentional or a defect.

### Patch

The LayerZero team acknowledged the finding as a feature to be implemented, committing to adding tests to address the issue.

© 2026 Otter Audits LLC. All Rights Reserved.

## Root Invocation Binding Should Always Be Included OS-LZO-SUG-02

### Description

If the signed payload does not always include the top-level/root call that initiated the authorization flow, then the same approved Merkle leaf may be consumed by a different call path or callee structure than the one the signers intended.

### Remediation

OneSig's `__check_auth` now only permits a single root-invocation call, ensuring the signed payload can be atomically executed.

© 2026 Otter Audits LLC. All Rights Reserved.

# A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

**CRITICAL**

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

**HIGH**

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

**MEDIUM**

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

**LOW**

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

**INFO**

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2026 Otter Audits LLC. All Rights Reserved.

# B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2026 Otter Audits LLC. All Rights Reserved.